Threads & Mutual Exclusion

Systems Programming — Mutex Complete Reference

EXAM CHEAT SHEET · Beginner Friendly · Code + Concepts

01 The Core Problem — Why Mutexes Exist

■ What Goes Wrong Without Synchronisation?

When multiple threads **share a variable**, they each read-modify-write it independently. Because the CPU can switch between threads at *any* instruction, the final result is unpredictable — this is called a **race condition**.

Thread 1	Thread 2
`/* counter = 0 */`	`/* counter = 0 */`
`tmp = counter → 0`	
	`tmp = counter → 0`
`tmp = tmp + 1 → 1`	`tmp = tmp + 1 → 1`
`counter = tmp → 1`	`counter = tmp → 1`

■ ■ Expected result = 2 but got 1. Both threads read counter=0 before either wrote back. This is a RACE CONDITION.

■ Key Vocabulary

Race Condition	Two threads access shared data concurrently and the outcome depends on scheduling order.

Critical Section	The block of code that accesses/modifies shared resources. Only one thread should run it at a time.
Mutual Exclusion	The guarantee that only ONE thread is inside a critical section at any moment.
Lock / Mutex	An OS-provided token. A thread must acquire it to enter the critical section; others wait.
Deadlock	Two or more threads each wait for a lock the other holds — everyone waits forever.

02 How the Mutex Protocol Works

A mutex enforces a simple rule: **lock before entering, unlock after leaving**. If the mutex is already locked, the thread **sleeps** (blocks) until it is free.

Step	Thread Action	Mutex State	Other Threads
1	Call pthread_mutex_lock()	`FREE` → `LOCKED`	Can proceed normally
2	Execute critical section	`LOCKED`	Block (sleep) if they also call lock()
3	Call pthread_mutex_unlock()	`LOCKED` → `FREE`	One sleeping thread wakes up
4	Continue after critical §	`FREE`	Winner re-runs from step 1

✔✔ Lock and Unlock ALWAYS come in pairs. Every lock() must have exactly one matching unlock().

03 pthread Mutex API — Every Function Explained

■ 1. Declare the Mutex (shared variable)

A mutex is just a variable of type **pthread_mutex_t**. It must be **visible to all threads** that need to share it — typically declared globally or inside a shared struct.

```
#include <pthread.h>


pthread_mutex_t myMutex; /* global — shared by all threads */
```

■ 2. pthread_mutex_init() — Initialise

Before using a mutex you must **initialise** it so the OS can allocate internal resources. Always pass **NULL** as the second argument unless you specifically need custom attributes.

```
int pthread_mutex_init(

pthread_mutex_t *mutex, /* pointer to your mutex */

const pthread_mutexattr_t *attr /* use NULL for defaults */

);


/* Returns: 0 on success, non-zero on error */


/* Example — mutex inside a struct: */

typedef struct {

pthread_mutex_t lock;

int value;

} Counter;


Counter* makeCounter() {

Counter* c = malloc(sizeof(Counter));

pthread_mutex_init(&c->lock, NULL); /* <-- init! */

c->value = 0;

return c;
```

```
}
```

■ 3. pthread_mutex_destroy() — Clean Up

When you are **finished** with a mutex, destroy it to release OS resources. Always destroy before freeing the memory that holds the mutex.

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);


/* Example: */

void freeCounter(Counter* c) {

pthread_mutex_destroy(&c->lock); /* destroy FIRST */

free(c); /* then free memory */

}
```

■ 4. pthread_mutex_lock() — Enter Critical Section

Acquires the mutex. If another thread already holds it, the calling thread **blocks (sleeps)** until the mutex is released. This is a **blocking** call.

```
int pthread_mutex_lock(pthread_mutex_t *mutex);


/* Returns 0 on success. Non-zero = error (something went very wrong) */


/* Example: safely increment a shared counter */

void increment(Counter* c) {

pthread_mutex_lock(&c->lock); /* LOCK — only 1 thread here */

c->value = c->value + 1; /* critical section */

pthread_mutex_unlock(&c->lock); /* UNLOCK — others may proceed */

}
```

■ 5. pthread_mutex_unlock() — Leave Critical Section

Releases the mutex. One waiting thread (if any) is woken up. Only the thread that **locked** the mutex should unlock it.

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

```
/* Returns 0 on success */
```

■ 6. pthread_mutex_trylock() — Non-Blocking Attempt

Like lock() but **does not block**. If the mutex is free it locks and returns 0. If it is already locked it returns an **error code immediately**.

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);


/* Returns 0 → successfully locked (you own it now) */

/* Returns EBUSY → mutex was already locked, try again later */


/* Example use-case: do other work if resource is busy */

if (pthread_mutex_trylock(&mutex) == 0) {

/* got the lock — do the work */

doWork();

pthread_mutex_unlock(&mutex);

} else {

/* locked by someone else — do something else instead */

doAlternativeWork();

}
```

■ Quick API Reference

Function	Returns	Description
`pthread_mutex_init(m, NULL)`	`int (0=ok)`	Initialise mutex before use
`pthread_mutex_destroy(m)`	`int (0=ok)`	Free OS resources when done
`pthread_mutex_lock(m)`	`int (0=ok)`	BLOCKING acquire — sleeps if locked
`pthread_mutex_unlock(m)`	`int (0=ok)`	Release mutex, wake a waiter
`pthread_mutex_trylock(m)`	`int (0=ok)`	NON-BLOCKING acquire — returns EBUSY if busy

04 Golden Rules — Memorise These!

✔ ONE mutex per shared resource	Each variable (or struct) that needs protection gets its OWN mutex. One mutex protecting two unrelated things is fine but creates unnecessary contention.
✔ Always init before use, destroy when done	Using an uninitialised mutex causes undefined behaviour. Failing to destroy causes resource leaks.
✔ Lock and Unlock in every code path	If your critical section has an early return or exception, make sure unlock() is still called. Missing an unlock() causes a deadlock.
✔ Keep critical sections SHORT	Locking is expensive (OS context switch). The longer a mutex is held, the more threads are blocked waiting. Move everything possible outside the lock/unlock pair.
✔ Never lock the same mutex twice from the same thread (default type)	Trying to lock a mutex you already hold will deadlock (unless RECURSIVE type).
✔ Only the locking thread should unlock	Unlocking a mutex from a different thread is undefined behaviour (default type).

05 Complete Code Example — Thread-Safe Counter ADT

This is the classic exam pattern: package a **mutex + data together in a struct**, then provide functions that always lock before touching the data.

```
#include <pthread.h>

#include <stdlib.h>


/* ■■ 1. Define the struct ■■■■■■■■■■■■■■■■■■■■■■■■■■ */

typedef struct {

pthread_mutex_t lock; /* the guard */

int value; /* shared data */

} Counter;


/* ■■ 2. Constructor ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

Counter* counter_create() {

Counter* c = malloc(sizeof(Counter));

pthread_mutex_init(&c->lock, NULL);

c->value = 0;

return c;

}


/* ■■ 3. Destructor ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

void counter_destroy(Counter* c) {

pthread_mutex_destroy(&c->lock);

free(c);

}
```

```
/* ■■ 4. Operations ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

void counter_increment(Counter* c) {

pthread_mutex_lock(&c->lock); /* LOCK */

c->value++; /* work */

pthread_mutex_unlock(&c->lock); /* UNLOCK */

}


void counter_decrement(Counter* c) {

pthread_mutex_lock(&c->lock);

c->value--;

pthread_mutex_unlock(&c->lock);

}


void counter_reset(Counter* c) {

pthread_mutex_lock(&c->lock);

c->value = 0;

pthread_mutex_unlock(&c->lock);

}


int counter_get(Counter* c) {

pthread_mutex_lock(&c->lock);

int val = c->value;

pthread_mutex_unlock(&c->lock);

return val;

}
```

✔✔ Notice: every single function that touches c->value calls lock() first and unlock() last. This is the fundamental pattern.

06 Locking Granularity — One Lock vs Per-Element Lock

When protecting a **data structure** (e.g. an array) you have a choice: one lock for everything, or one lock per element. This trade-off comes up constantly in exam questions.

Option 1 — Single Lock (Coarse-Grained)

Lock the entire structure before accessing any element.

```
typedef struct { pthread_mutex_t lock; int data[100]; } SafeArray;
```

✔ Simple to implement

✔ Easy to reason about

✔ Low memory overhead

✗ Bottleneck: all threads serialised

✗ Poor concurrency for large arrays

Option 2 — Per-Element Lock (Fine-Grained)

Each element has its own mutex; threads only block each other on the same index.

```
typedef struct { pthread_mutex_t lock; int value; } SafeElement; SafeElement arr[100];
```

✔ Threads access different elements in parallel

✔ Much better concurrency

✗ More memory (1 mutex per element)

✗ Complex — deadlock risk with multiple locks

✗ Each access: lock + unlock overhead

■ Exam Tip: The right choice depends on the workload. If threads rarely access the same index, fine-grained is faster. If you need simplicity or low memory, coarse-grained is safer.

07 Linked List with Mutex — Exam Favourite

Concurrent linked list access is a classic exam problem. Here are the key operations and both locking strategies.

■ Node Structure

```
typedef struct list_node_s {

int data;

struct list_node_s* next;

pthread_mutex_t mutex; /* for fine-grained locking */

} LNode;
```

■ member() — Check if Value Exists

```
int member(int value, LNode* head_p) {

LNode* curr_p = head_p;

/* Walk list: stop if NULL or found a node >= value */

while (curr_p != NULL && curr_p->data < value)

curr_p = curr_p->next;

if (curr_p == NULL || curr_p->data > value)

return 0; /* not found */

return 1; /* found */

}
```

■ Solution 1 — Single Mutex for Whole List (Simple)

```
pthread_mutex_t list_mutex; /* one global lock */



/* Usage: */

pthread_mutex_lock(&list_mutex);

member(value, head_p); /* or insert / delete */
```

```
pthread_mutex_unlock(&list_mutex);


/* ISSUE: all threads serialised — no concurrent reads allowed */
```

■ Solution 2 — One Mutex Per Node (Complex but Faster)

Use **hand-over-hand locking** (lock the next node before unlocking the current one so no other thread can sneak in):

```
/* Lock head_p first, then walk node by node */

pthread_mutex_lock(&head_p_mutex);

curr_p = head_p;

while (curr_p != NULL && curr_p->data < value) {

if (curr_p->next != NULL)

pthread_mutex_lock(&curr_p->next->mutex); /* lock next */

if (curr_p == head_p)

pthread_mutex_unlock(&head_p_mutex); /* release head */

else

pthread_mutex_unlock(&curr_p->mutex); /* release curr */

curr_p = curr_p->next;

}

/* ... check curr_p and unlock remaining ... */
```

■ ■ Fine-grained linked list locking is complex. Using more than one lock simultaneously creates DEADLOCK risk. The extra complexity is only worth it for high-contention lists.

08 Mutex Types (pthread_mutexattr_t)

You can change mutex behaviour by passing a **pthread_mutexattr_t** to init(). Most code uses NULL (default). Know the four types for exams.

PTHREAD_MUTEX_NORMAL

Default / basic type

- No deadlock detection
- Re-locking it from the same thread → DEADLOCK (hangs forever)
- Unlocking from wrong thread → undefined behaviour
- Unlocking when already unlocked → undefined behaviour

PTHREAD_MUTEX_ERRORCHECK

Adds error checking

- Re-locking from same thread → returns error (no deadlock)
- Unlocking from wrong thread → returns error
- Unlocking when unlocked → returns error
- Safer for debugging but slightly slower

PTHREAD_MUTEX_RECURSIVE

Allows same thread to lock multiple times

- Same thread can call lock() N times without deadlock
- Must call unlock() exactly N times to fully release
- Unlocking from wrong thread → returns error
- Useful for recursive functions that need the lock

PTHREAD_MUTEX_DEFAULT

Implementation-defined (usually same as NORMAL)

- Recursive lock → undefined behaviour
- Wrong-thread unlock → undefined behaviour
- Safest to treat as NORMAL unless docs say otherwise

■ How to Set a Mutex Type

```
pthread_mutexattr_t attr;

pthread_mutexattr_init(&attr);

pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_ERRORCHECK);


pthread_mutex_init(&myMutex, &attr); /* pass attr instead of NULL */


pthread_mutexattr_destroy(&attr); /* clean up the attribute */
```

09 Deadlock — How It Happens & How to Avoid It

Deadlock = two (or more) threads each hold a lock and wait for the other's lock. Nobody proceeds. The program hangs forever.

Thread 1	Thread 2
`pthread_mutex_lock(&A;);`	`pthread_mutex_lock(&B;);`
`/* doing work... */`	`/* doing work... */`
`pthread_mutex_lock(&B;);`	`pthread_mutex_lock(&A;);`
`/* BLOCKED — B held by T2 */`	`/* BLOCKED — A held by T1 */`
`/* ... waiting forever */`	`/* ... waiting forever */`

✕✕ Both threads sleep waiting for the other. Nobody unlocks. DEADLOCK.

■ How to Prevent Deadlock

Lock Ordering	Always acquire multiple locks in the SAME order in every thread. If all threads lock A before B, deadlock cannot occur.
Try-Lock + Back-Off	Use trylock(). If you can't get the second lock, release the first and retry.
Minimise Locks Held	Hold as few locks as possible for as short a time as possible.
Avoid Nested Locks	If you can redesign to never hold two locks at once, deadlock is impossible.

10 Amdahl's Law — Speedup Limits

Not all code can be parallelised. Amdahl's Law tells you the **maximum speedup** you can get from parallelising a fraction of your program.

$$\text{Speedup} = \frac{1}{(1 - p) + p/n}$$

Variable	Meaning	Example
`p`	Fraction of code that CAN be parallelised (0 to 1)	0.8 means 80% parallelisable
`1-p`	Fraction that MUST run serially (never speeds up)	0.2 means 20% always serial
`n`	Number of CPU cores / threads	4 cores
`Speedup`	How many times faster than single-threaded	Maximum gain achievable

■ Key insight: If 20% of your code is serial (p=0.8), you can NEVER get more than 5× speedup no matter how many cores you add. Locking creates serial bottlenecks — keep critical sections short!

11 Exam Quick-Fire Q&A

Q1. What is a mutex?

→ A mutual-exclusion lock provided by the OS. At most one thread can hold it at any time.

Q2. What happens when a thread calls lock() on an already-locked mutex?

→ The thread BLOCKS (sleeps) until the mutex is released by the holder.

Q3. What does trylock() return if the mutex is busy?

→ A non-zero error code (EBUSY). It does NOT block — it returns immediately.

Q4. Why must lock/unlock be in pairs?

→ A missing unlock means the mutex is held forever → deadlock for all waiting threads.

Q5. Why keep critical sections short?

→ Locking forces other threads to wait. Long critical sections = poor concurrency + big overhead.

Q6. How many mutexes per resource?

→ ONE mutex per independent shared resource (or group of fields that are always accessed together).

Q7. What is deadlock?

→ Thread A holds lock X, waits for Y. Thread B holds Y, waits for X. Both sleep forever.

Q8. What mutex type allows the same thread to lock multiple times?

→ PTHREAD_MUTEX_RECURSIVE. Must call unlock() the same number of times.

Q9. Coarse vs fine-grained locking?

→ Coarse = one lock, simple, less concurrency. Fine = per-element lock, complex, more concurrency, deadlock risk.

Q10. What must you do before free()-ing a struct that contains a mutex?

→ Call pthread_mutex_destroy() first, THEN free().

Good luck on your exam! Remember: lock → critical section → unlock. Always init. Always destroy. Keep it short.